Determinants of Speculative Decoding Efficiency in Resident MoE Inference

Working research manuscript

October 16, 2026

Abstract

Speculative decoding trades fewer target iterations for more work per iteration. Prior work establishes how expert activation and batch size affect this tradeoff in mixture-of-experts (MoE) inference. We characterize additional determinants in a resident single-GPU engine: draft availability, actual verification shape, routing dependence, and the service measurement boundary. A BF16 OLMoE/n-gram experiment covers seven batch sizes and eight draft caps with two repetitions per cell. An ordinary-decoding routing model predicts distinct experts in held-out speculative executions with mean absolute error 1.91 using actual verification rows, versus 15.30 using the nominal batch–cap product. A separate single-layer replay supports a distinct-expert cost fit with 2.35% held-out mean absolute percentage error; its compute branch is not identified. Instrumented compiled execution also shows that padding can enlarge physical expert participation beyond valid-row occupancy. Small-batch throughput gains coexist with nearly neutral or negative finite-wave throughput at larger batches, while request-level effective token latency can improve more. Energy and proposal coverage expose further differences between caps. Eager service results retain cold-shape and run-order effects; compiled service performance, neural drafters, and controller benefit remain unmeasured.

1 Introduction

The practical decision in speculative decoding is how much drafting to perform before asking the target model to verify it. For resident MoE inference, the extra positions can touch experts that ordinary decoding would not use in that iteration. The resulting working-set expansion competes with the benefit of emitting multiple tokens.

MoESD already explains the moderate-batch opportunity using expert activation and compute saturation, and demonstrates it experimentally [2]. Our study asks a narrower question: *which observed determinants are needed to relate a configured draft cap to expert participation, useful token progress, and measured service cost?* A qualitative batch reversal alone is not our proposed novelty. In particular, a configured cap does not guarantee that an n-gram proposer supplies that many tokens, nor that every scheduled request remains active for the full measurement window.

The current contribution is an exploratory characterization connecting the executed verification shape to logical expert participation and documenting how draft coverage and measurement objectives constrain interpretation of the batch–cap surface. The implementation also supports a step-level controller for testing whether these observations improve decisions. A future method claim requires complete-cost improvement over tuned native batch schedules. Occupancy prediction alone does not establish a latency mechanism, and the current characterization does not identify a dominant fixed-cost term.

Evidence status. Section 4 reports all 56 cells and 112 performance records of the eager r02 campaign, separate route diagnostics, and the r03 component replay. There are two service repetitions per cell, not 112 independent confirmations of one effect. All speculative lengths are held out from the route-probability fit, but prompts are shared across fit and test. Compiled routes are measured with instrumentation; compiled service performance, controller comparisons, and neural-drafter experiments remain pending. Section 3.4 is hypothetical arithmetic, not a hardware result.

2 Background and Scope

We consider one GPU with all target expert weights resident in device memory. Expert offloading, cache replacement, and inter-GPU communication are outside the principal comparison. A sequence supplies up to γ draft tokens; the unchanged target verifier and sampler determine the accepted prefix and next emitted token. $\gamma = 0$ denotes ordinary decoding. We first use n-gram drafting, then a compatible neural drafter where available. EAGLE-3 is a candidate draft source [3, 6]; each evaluated checkpoint must match the target, tokenizer, and engine interface.

Time and population boundaries. The current experiment controls the initial synchronous wave size and

records actual active batches as requests finish. Each wave drains before the next starts; vacant slots are not refilled. Complete-population throughput includes request submission, prefill, and decoding, so it measures a finite burst rather than steady-state serving capacity. Request-level effective token latency uses the interval between first and last emitted chunks divided by output tokens minus one. Natural-generation evaluation separately reports capped, failed, and completed requests. Fixed-length runs control output length, not task quality or natural generation workload.

What a working set measures. A union of selected expert IDs describes logical expert participation. It is not a hardware memory-traffic counter: caches, tensor tiles, quantization metadata, padding, and kernel scheduling can change the physical bytes read. The purpose of calibration is to determine when this observable predicts time, and where a richer cost term is required.

3 Model and Predictions

3.1 Expert occupancy with top-k routing

For layer ℓ, let $R_{\ell,s}$ be the distinct expert IDs selected for valid token slot s, E_ℓ the routed expert count, and $k_\ell = |R_{\ell,s}|$ when top-k is fixed. Define the per-token inclusion probability

$$q_{\ell,e} = \Pr(e \in R_{\ell,s}), \qquad \sum_{e=1}^{E_\ell} q_{\ell,e} = k_\ell. \tag{1}$$

Linearity of expectation yields the exact identity

$$\mathbb{E}[D_\ell(S)] = \sum_e \left[1 - \Pr\left(\bigcap_{s=1}^{S} \{e \notin R_{\ell,s}\}\right)\right]. \tag{2}$$

If token slots are independent and have stationary inclusion probabilities, this becomes

$$\widehat{D}_\ell(S) = \sum_e [1 - (1 - q_{\ell,e})^S]. \tag{3}$$

No independence among the experts selected by one token is required. If a trace instead reports the fraction $p_{\ell,e}$ of all assignments, with $\sum_e p_{\ell,e} = 1$, then $q_{\ell,e} = k_\ell p_{\ell,e}$. Treating top-$k$ as Sk independent draws with replacement gives a different model and can violate the exact $D_\ell(1) = k_\ell$ check.

Let $m_{t,i}$ be the draft length available for request i before truncation and $k_{t,i}$ the prefix actually scheduled for verification. For an unchunked decode-only call, $S_t = \sum_{i=1}^{B_t}(1 + k_{t,i})$, with $k_{t,i} \le \min(m_{t,i}, \gamma_t)$. Thus $B(\gamma + 1)$ is only a nominal upper shape, attained when every request supplies and schedules the full prefix. The primary measurement is the valid token-row count at the executed target call. Chunking, output limits, mixed prefill/decode, and padding are recorded separately; padded rows do not enter S_t. Direct trace unions measure D_ℓ and expose correlations among tokens and requests. Uniform routing is a reference curve, not the source of final measured predictions.

Graph execution may process a larger physical row count $P_t \ge S_t$, excluding padding internal to expert matrix-multiplication tiles. Instrumentation distinguishes valid, padding-only, and total physical expert histograms and their unions; padding can add expert participation without useful token progress. Section 4.7 measures this distinction in an instrumented compiled diagnostic. Neither the eager occupancy result nor the compiled physical-row union measures graph-padding time or hardware memory traffic.

3.2 From occupancy to complete iteration cost

Let $w_{\ell,e}$ be expert weight storage bytes in the deployed representation. The logical routed working-set estimate is

$$\widehat{W}_\ell(S) = \sum_e w_{\ell,e}[1 - (1 - q_{\ell,e})^S]. \tag{4}$$

Shared experts are recorded separately. For a fixed layer, precision, and expert kernel, the proposed component fit is

$$\widehat{T}_{\mathrm{MoE}}(D, S) = \max(aD, bS) + c, \qquad a, b, c \ge 0. \tag{5}$$

Here D and S are measured distinct experts and actual valid rows in the eager component replay. A compiled fit would need to distinguish physical rows and their expert union. The coefficients are local empirical costs, not measured bandwidth, FLOP rates, or an identified decomposition of runtime overhead. The max-form is a hypothesis; expert imbalance, alignment, cache state, and launch behavior may require additional terms. The measured r03 fit selects the $aD + c$ boundary over its sampled domain (Section 4.6); this establishes neither zero compute cost nor full-model fixed-cost dominance.

The component experiment replays routes with real BF16 expert weights and synthetic activations through the selected expert implementation, including preparation, alignment, matrix multiplication, activation, and reduction. Histogram-only replay preserves expert counts but loses token co-occurrence. Calibration batches are $\{1, 4, 16, 64\}$ and held-out batches $\{2, 8, 32\}$. A fit using measured D tests the D, S-to-time relation; the occupancy experiment separately tests Equation 3's route-to-D prediction. Neither validates complete-step time by itself.

Attention depends on context length, query count, and backend. Sharing logical KV data does not imply constant physical reads or attention computation. Complete iteration costs therefore come from a separate table indexed by batch, scheduled cap, production budget, and execution regime. The current synchronous host-step

boundary includes target verification and production of the next draft, plus sampling, scheduling, and control. We do not subtract guessed component times or add overlapping phases. A component fit explains only its declared interval; full-population throughput and energy remain separate measurements.

3.3 Accepted work and the decision rule

Let M be the available draft length before truncation under production budget G. For a chosen cap $\gamma \leq G$, distinguish availability $v_j = \Pr(M \geq j)$ from conditional prefix survival $s_j = \Pr(\text{first } j \text{ accepted} \mid M \geq j)$, for $j \leq \gamma$. These probabilities refer to the same state and budget; their product requires no independence assumption. With one correction/bonus token and no terminal or output-budget censoring, expected emitted tokens per request per round are

$$A(\gamma; G) = 1 + \sum_{j=1}^{\gamma} v_j s_j. \tag{6}$$

Empty n-gram proposals contribute zero to every v_j but still incur lookup and target-step costs. Availability is measured before controller truncation, including empty proposals; ranks beyond the production budget are unobserved. Conditional survival uses requests for which rank j was scheduled, counting an earlier rejection as failure of that prefix. Positions after the first rejection are not additional tested rejections when estimating a per-position acceptance probability. EOS and output-cap censored observations are excluded from this uncensored estimator and reported in the generation population.

Only with full availability and constant conditional acceptance α does Equation 6 reduce to $1+\alpha+\cdots+\alpha^{\gamma}$. The scalar estimate is $\hat{\alpha} = C_a/C_n$: for an exposed step with a accepted and n tested positions, $C_a \leftarrow 0.9C_a + 0.1a$ and $C_n \leftarrow 0.9C_n + 0.1n$, initialized to $(C_a, C_n) = (0.7, 1)$. Steps without tested positions neither update nor decay these states. This is a ratio of EWMA counts, rather than an EWMA of step-level ratios; decay is per eligible step, not per token or elapsed time. Separate availability and prefix-survival EWMAs retain eligible-step weighting of proportions, not exposure-count weighting across steps. Unseen survival ranks use a geometric prior and unseen availability starts at one, both unvalidated. Rank-population changes and monotone projection can introduce approximation error; we check useful progress separately.

For a stable batch and matched ordinary-decoding baseline,

$$\widehat{\text{Speedup}}(B, \gamma) = \frac{\widehat{A}(\gamma; G)\widehat{T}_{\text{round}}(B, 0; 0)}{\widehat{T}_{\text{round}}(B, \gamma; G)}. \tag{7}$$

The online action is selected from a small supported set Γ including zero:

$$\gamma_t^* = \arg\max_{\gamma \in \Gamma} \frac{\widehat{A}_t(\gamma; G)}{\widehat{T}_{\text{round}}(B_t, \gamma; G, L_t)}. \tag{8}$$

The policy uses recent observed availability and conditional survival with an offline cost table. All fields are available before the selected action takes effect; shorter actions break ties. Native production uses the selected budget, whereas the initial post-proposal integration holds G fixed and includes its full drafting cost, even when $\gamma = 0$. Context dependence can be fixed within an experiment or represented by calibration bins. This implemented decision rule is an instrument for testing the measured determinants; a service benefit over native batch schedules remains to be established.

3.4 Illustrative arithmetic, not results

For a hypothetical uniform layer with $E = 64$, $k = 8$, $\gamma = 4$, full proposal availability, and $\alpha = 0.7$, Equation 3 gives $D(1) = 8$ and $D(5) \simeq 31.17$, while $A \simeq 2.773$. If expert weight reads alone determine time and all other costs vanish, Equation 7 gives approximately $0.71\times$ at $B = 1$ and $1.83\times$ at $B = 8$. These values illustrate a possible change of sign. They do not establish a measured turning point. Saturation of expert participation does not remove compute growth, drafting costs, or attention costs; speedup need not approach A at arbitrarily large batches.

Testable predictions. In the bandwidth-sensitive regime, larger ordinary-decoding working sets should reduce the *relative* verification expansion. At equal batch and draft length, observed routing concentration can alter this expansion. If fixed or attention costs dominate, the working-set ratio alone should overpredict its effect on total time. F1 tests occupancy; the component replay tests timing. Separate held-out configurations must test whether complete calibrated costs rank candidate lengths correctly.

4 Measured Characterization

4.1 Setting and population

The completed `olmoe_ngram_matrix_r02` campaign uses OLMoE-1B-7B-0924-Instruct at revision `7f1c97f4`, BF16 resident weights, and vLLM 0.26.0 on an NVIDIA RTX PRO 6000 Blackwell Server Edition. Execution is eager, the configured MoE backend is Triton, attention is automatically selected, and the KV-cache allocation is 6 GiB. Each of 56 cells combines $B \in \{1, 2, 4, 8, 16, 32, 64\}$ with $\gamma \in \{0, \ldots, 7\}$ and has two performance repetitions. Every repetition completes the same 64 ShareGPT-derived requests with 128 forced-output tokens each, or 8,192

output tokens. Detailed route capture is performed in separate diagnostic episodes. The campaign held the shared GPU lock for 3,091.24 seconds, including initialization, warmup, and diagnostics.

Only $B = 64$ was warmed before measurement. Cold compilation for other shapes and run-order variation are retained. The plots show ratios of run medians, preserving the two raw values in the accompanying data; with two values the median equals their mean. These exploratory observations do not establish statistical significance, a universal crossover, or the effect of compiled execution. The timing window sums drained synchronous waves and includes submission, prefill, and decode. It is a finite-population burst measurement, not a continuously replenished serving workload.

4.2 Actual shape predicts participation better than the configured cap

For each layer, we fit token-weighted inclusion probabilities from $\gamma = 0$ routes across all seven batches. We then predict distinct experts for $\gamma = 1$–7 using actual target verification histograms. Prefill and mixed steps are excluded from this decode-only analysis; rejected draft positions remain included. All retained histograms satisfy the top-k count invariant. The 3,472 fit and 23,568 test layer-step records share prompts and are correlated, so their count is not an independent-system sample size.

With actual S, mean absolute error is 1.9145 experts and RMSE is 3.1453 (Figure 1). Using nominal $B(\gamma+1)$ with the same q_e raises MAE to 15.2994; using actual active batch but still the nominal cap gives 13.6975. Thus correcting only the batch tail does not recover the executed verification shape. This comparison isolates the choice of shape input, not the benefit of nonuniform probabilities over a uniform model. It validates an occupancy approximation in these held-out cap executions; it does not yet validate kernel time, action rankings, or an independent workload.

Residuals are structured. The first decode step contributes 784 of 23,568 observations (3.33%) but 25.78% of squared prediction error. Its MAE is 5.73 experts, versus 1.78 later. All points remain in the main result. AR itself exhibits this concentration: at layer zero and $B = 32$, the first-step union is 26 experts, versus 61.07 on average afterward. The first decode step processes the first generated token, not directly a template token. Shared templates or repeated token identities are possible explanations to test, not established causes; the anomaly is not specific to speculative drafts.

4.3 Throughput and request token latency answer different questions

At $B = 1$, all speculative caps improve median finite-wave throughput by 1.129–1.179× over AR (Figure 2). At $B = 2, 4$, the ranges are 1.061–1.107× and 1.025–1.066×. Every speculative cell at $B = 16, 32$ is at or below AR in the unrounded ratios; at $B = 64$ the range is 0.995–1.028×. These observations differ from a prediction of a universal small-batch loss followed by a moderate-batch win, but do not contradict a model whose workload, draft availability, kernel regime, and cost assumptions differ.

Table 1: Different objectives on the same r02 executions. Ratios compare two-run medians with matched AR. Effective-TPOT speedup is AR time divided by speculative time; throughput speedup is speculative rate divided by AR rate.

B	γ	Wave throughput	Effective TPOT
16	6	0.9999×	1.1299×
64	6	1.0234×	1.1582×

Request-level effective token latency can improve more than complete-wave throughput. We average each request's first-to-last-chunk interval divided by 127 output-token intervals within a run, then take the run median. Table 1 illustrates the difference. Requests that finish early leave unfilled slots while a wave waits for its last request.

A receipt-time decomposition separates mean first receipt, mean post-first generation span, and the barrier term $\max_i(\text{last}_i) - \text{mean}_i(\text{last}_i)$. Summing across waves for 64 requests and averaging the two runs, $\gamma = 6$ versus AR at $B = 16$ reduces the mean-span term by 0.70788 s but increases the barrier term by 0.71303 s; the first-receipt term decreases by 0.00436 s. At $B = 64$, the corresponding changes are −0.25573, +0.20864, and +0.000305 s. Thus increased initial-receipt time does not explain this gap. The observed barrier contribution is not a recoverable-time bound: refilling requests would change batches and execution. A matched queued-versus-wave experiment remains pending, as does attribution to particular kernels or fixed costs.

Raw repetition values remain important: at $B = 1$, AR throughput is $(82.55, 87.75)$ tokens/s versus $(95.69, 96.63)$ for $\gamma = 1$; at $B = 64$, the corresponding pairs are $(4420.34, 4091.67)$ and $(4211.59, 4299.17)$. A cell near unity cannot be interpreted independently of this run variation and incomplete warmup.

4.4 Draft availability and progress

The full performance population includes empty proposals and shrinking wave tails (Figure 3). At $B = 1$, the fraction of decode request-steps with nonempty proposals is 18.07%, 14.09%, and 13.37% for $\gamma = 1, 4, 7$. The corresponding mean available draft-slot counts are 0.181, 0.562, and 0.928 per request-step, while actual emitted tokens are 1.095, 1.149, and 1.158. These are full-population observations, not an acceptance rate conditioned on an n-gram hit. The accepted/proposed-slot ratio also is not

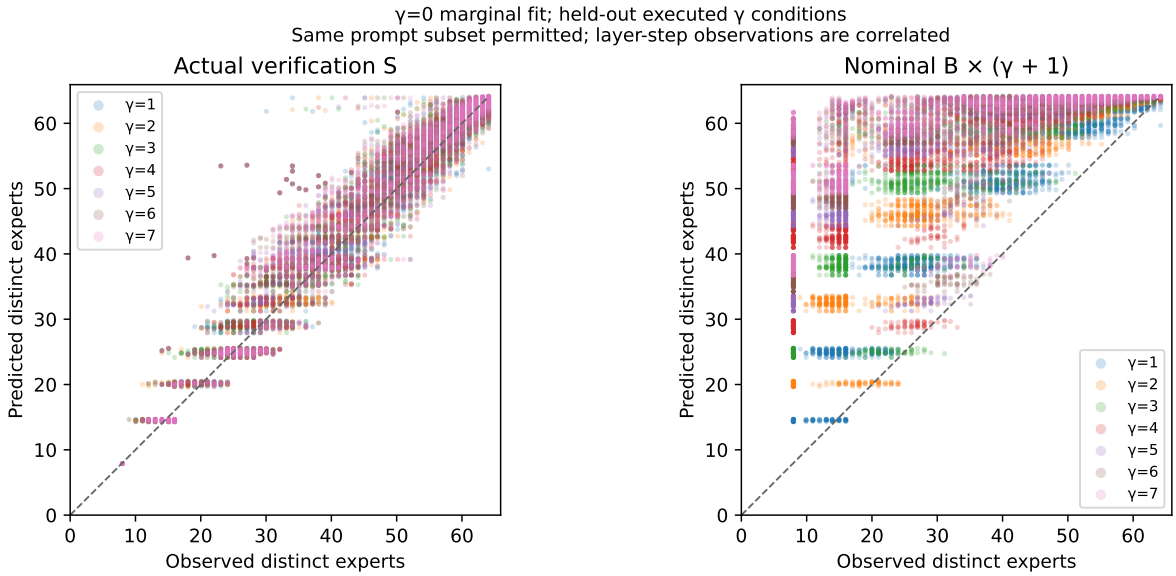


Figure 1: F1: expert-occupancy prediction using the same AR-fitted inclusion probabilities q_e in both panels. Left uses actual valid verification rows S; right uses nominal $B(\gamma+1)$. Fit: 3,472 AR layer-step observations. Test: 23,568 observations from $\gamma = 1$–7 executions, including rejected draft positions. The prompt subset is shared, so this is held-out execution-condition validation, not independent-workload validation. All eligible decode points, including the first step, remain in both panels. Logical expert counts do not measure HBM traffic or time.

the per-tested-position α in a geometric model.

This disparity explains why configured cap alone is a poor proxy for verification work and why a conditional acceptance estimate cannot determine useful progress without availability. It does not by itself explain the throughput curve: proposal lookup, target kernels, sampling, scheduling, and wave completion all remain in the cost. Terminal/output-limit steps can make one plus native accepted drafts differ from client-emitted tokens, so we retain both the proxy and actual emission measurements.

Offline replay with a cold prior per run illustrates the estimator distinction. At $B = 1$, $\gamma = 7$, changing the scalar estimate from an EWMA of step ratios to the ratio of EWMA counts changes tested-position-weighted pre-step bias from -0.2285 to -0.0137. Progress-prediction MAE remains approximately 0.282 tokens; availability and prefix weighting are unchanged. This is a statistical correction on fixed traces, not an executed adaptive policy or a performance gain. Both replay versions are retained.

4.5 Energy is a separate measured objective

Energy comes from the cumulative NVML device counter over each measured wave, with no idle-baseline subtraction. At $B = 1$, AR uses 1.4982 J/token; speculative caps use 1.3357–1.3728 J/token (Figure 4). At $B = 64$, AR uses 0.0650 J/token and speculative caps span 0.0638–0.0675 J/token. For example, $\gamma = 6$ at $B = 64$ has throughput ratio 1.0234 but uses 0.0666 J/token. The energy and throughput objectives therefore have different observed orderings; small differences still inherit the two-repeat and warmup limitations.

4.6 Distinct experts predict a measured layer interval

The r03 replay uses OLMoE layer 0's original resident BF16 weights through the configured vLLM `TritonExperts` implementation. Its 154 route snapshots preserve measured expert histograms, with synthetic token co-occurrence, unit-RMS normal activations, and uniform top-k weights. Actual rows span $S = 1$–148 and distinct experts $D = 8$–64. Snapshot selection uses routing quantiles and token-count extremes before inspecting times. Each snapshot receives ten warmup calls and five repeats of twenty timed calls; the reported time is the median of repeat means. A 256 MiB unrelated-buffer turnover occurs outside timers and does not reconstruct the full model's cache state.

The CUDA-event boundary includes preparation, alignment, the two expert matrix multiplications, activation, and reduction. It excludes routing, attention, drafting, and scheduling, and can include host launch gaps between kernels. Fitting the same 88 calibration snapshots with equal weights gives $a = 0.009471$ ms/expert and $c = 0.153261$ ms. Equation 5 selects its pure-D boundary, with the same predictions as $aD + c$. On the 66 held-out snapshots, MAE is 12.731 μs and mean absolute percentage error is 2.345%; the corresponding token-count-plus-intercept and constant fits give 21.673% and

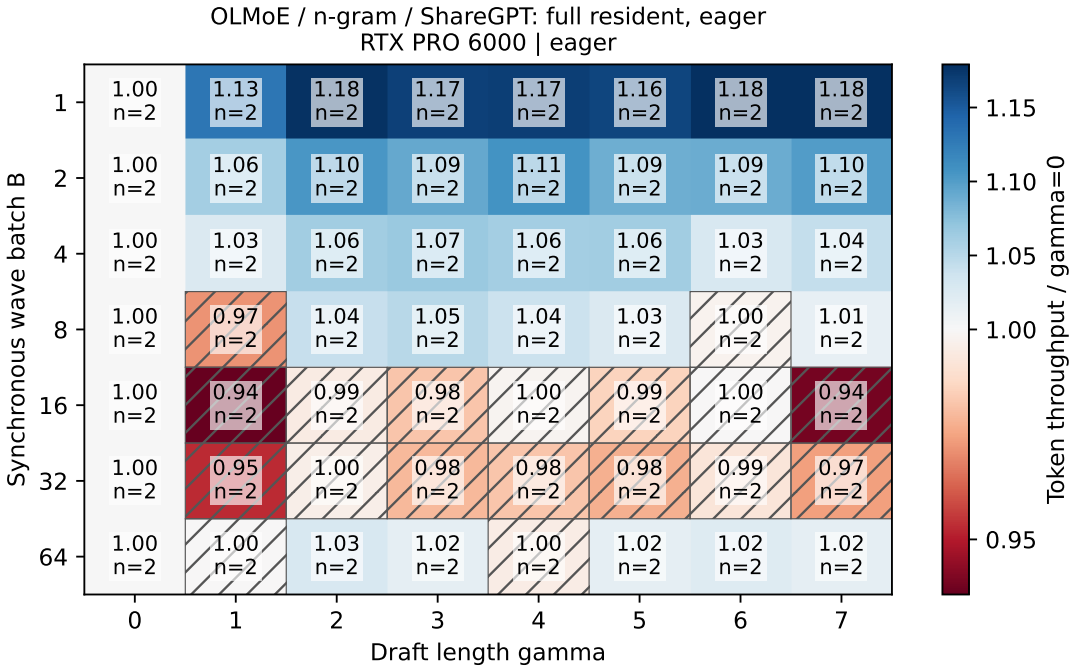


Figure 2: F2: complete r02 finite-wave throughput surface, normalized to AR at the same initial batch. Each of 56 cells has two repetitions. Windows include prefill and wave drain; hatched cells have an unrounded ratio below one. Cold-shape and order effects remain, so the surface is exploratory rather than a steady-state capacity or significance claim.

29.517% (Figure 5). All snapshots, including calibration outliers, remain included.

This result supports distinct experts as a useful local cost variable over the sampled shapes, not a measured HBM-byte model. The inactive bS branch leaves b unidentified over a range and does not identify a compute-saturation transition. The fitted c belongs only to this single-layer eager interval; it is not full-model c_0 and does not establish fixed-cost dominance. Separate single-call profiler kernel sums used a different cache condition without repeated turnover, so subtracting them from event intervals cannot isolate CPU overhead. Complete-step action ranking remains a separate test.

4.7 Physical participation under graph padding

A separate instrumented compiled diagnostic runs $\gamma \in \{0, 4\}$ at $B \in \{1, 8, 32\}$, each with 64 unique requests and 32 forced output tokens (Figure 6). All six route files cover every target layer and decode step, with exact top-k, histogram, valid-row, and physical-row conservation checks passing. Ordinary decoding uses FULL graph dispatch with $S = P$. With $\gamma = 4$, 1,569 of the 1,747 $B = 1$ decode calls have $S = 1$ but $P = 5$ under PIECEWISE dispatch; the remaining 178 have $S = P = 5$ under FULL dispatch. Mean layer-step expert participation rises from 9.62 on valid rows to 26.73 on physical rows. At $B = 8, 32$, all decode calls use PIECEWISE dispatch; 224/248 and 54/62 calls are padded, with respective valid-to-physical means 38.91 to 46.36 and 59.27 to 60.90.

The vLLM V1 graph-size construction rounds capture sizes to multiples of $\gamma + 1$ when full decode graphs are enabled; nonuniform decode calls reuse that size table. Thus valid-row occupancy can substantially understate the physical expert union for this graph configuration. Source inspection confirms that OLMoE already uses V1 and that enabling routed-expert return does not change its graph-size dispatch; the padding observer reads routing buffers after the call. The same configuration and shape therefore imply the same padding rule with tracing disabled, but measured expert unions and timings cannot be transferred directly. This diagnostic uses a scheduler cap of 32, whereas planned service runs use 64. It establishes physical participation under instrumentation, not HBM traffic, padding time, or compiled service benefit.

Testing the physical-row input. We fit layerwise q_e only on the compiled $\gamma = 0$ runs at $B = 1, 8, 32$ (5,952 tokens per layer), then test $\gamma = 4$ on the same prompt subset. At those three batches, Equation 3 using S predicts valid D with MAE 0.566, 3.438, and 1.946 experts.

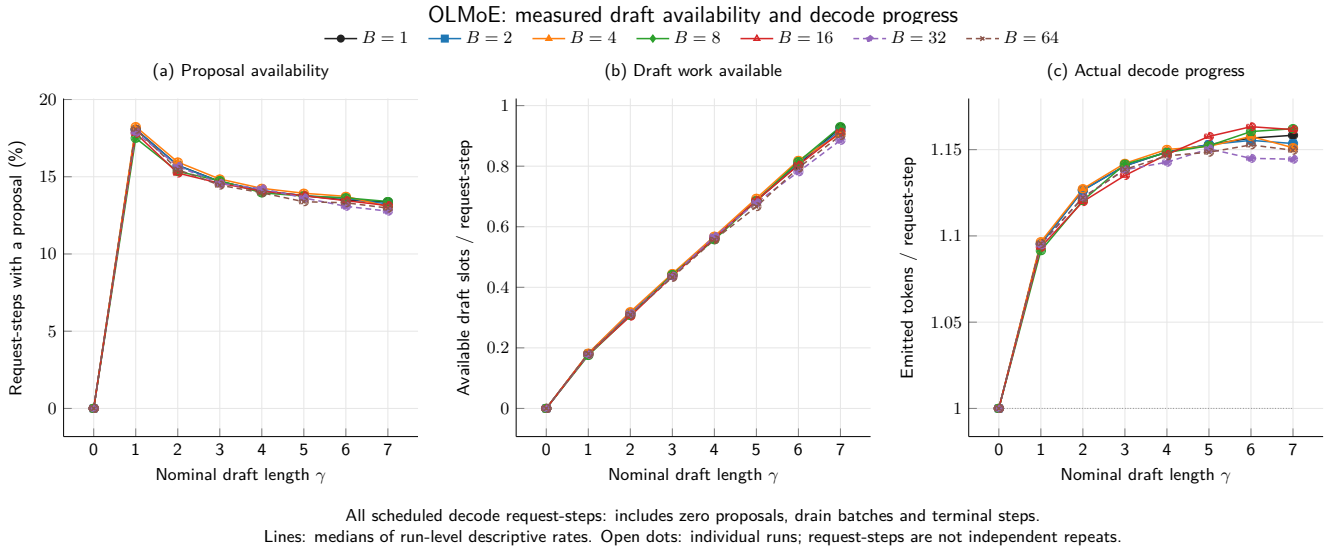


Figure 3: F5 coverage/progress component: nonempty proposal frequency, mean available draft slots, and actual client-emitted tokens per decode request-step. All 112 performance records contribute, including empty drafts, shrinking drain batches, and terminal steps. Lines summarize two run-level rates and open markers retain their raw values; request-steps are not independent repetitions. This is not a conditional-survival distribution or EWMA-validation plot.

For physical D, replacing S with P reduces MAE from 17.675/6.307/1.818 to 3.133/2.898/1.464. This is held-out execution-condition validation with correlated layer-steps, not independent-workload evidence. At $B = 1$, the P-based prediction still has bias +2.562 experts and MAPE 12.864%. All 21,920 adjacent padded layer-step pairs within a wave have identical recorded padding histograms. This temporal repetition alone does not test within-call independence; padding marginals also differ from AR, so treating padding as fresh independent AR tokens remains unvalidated. Histogram equality establishes neither identical ordered token routes nor cache hits.

Current interpretation and next construction. Actual verification shape is necessary for interpreting valid-row occupancy; physical graph padding adds a distinct execution condition. Measured expert participation predicts a single-layer eager interval, while service objectives still retain their population and time boundaries. The current data do not identify the dominant full-model cost or controller value. The next measurements compare uninstrumented compiled service with eager mode and test broader draft coverage. Natural-stop inspection and held-out native table/threshold comparisons then test task semantics and decision value.

5 Adaptive Draft-Length Design

5.1 Reuse the production execution path

vLLM 0.26.0 already supports a batch-to-length schedule through `num_speculative_tokens_per_batch_size` [8, 9]. Entries specify inclusive batch intervals and a draft cap, including zero; caps need not be monotone across intervals. The native scheduler changes the production budget, making this a strong deployable baseline. We freeze a table selected from calibration data, including an explicit rule for intermediate batches, then evaluate its actual execution. Static-cell scores used to select a table are not its measured service result. A controller switch retains the target computation and sampler.

The selected length must propagate consistently through proposal, scheduler reservations, verification slots, and the acceptance update. The event record includes proposed length, selected length, scheduled length, emitted tokens, rejection position, and decision time. Empty drafts, $\gamma = 0$, request completion, and batch changes are normal transitions. Causal step identifiers link proposals to verification; asynchronous feedback must follow that association. Online EWMA states persist across waves, batches, and repetitions within one engine; initial states are recorded. The per-run cold-prior offline replay is a separate diagnostic.

5.2 Draft truncation and overhead

The initial controller truncates an available prefix after a full-budget proposal and before scheduler reservation.

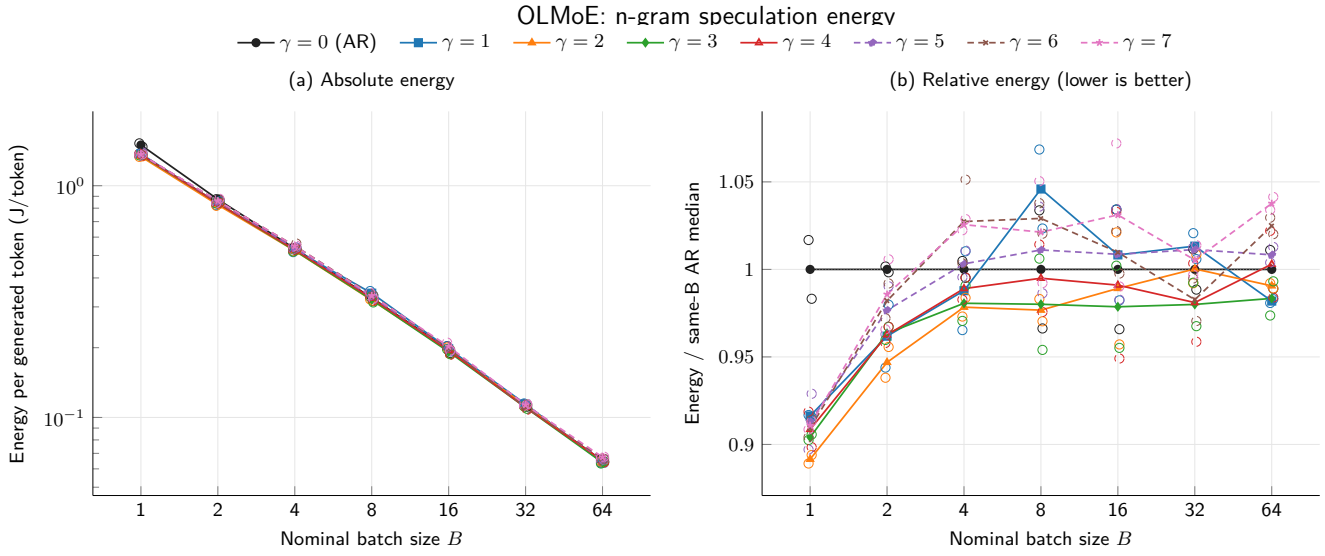


Figure 4: F4: GPU joules per emitted token over the complete wave windows, including prefill. Lines are two-run medians; open markers are raw runs. Right-panel points divide each run by the same-batch AR median, without paired-run inference, confidence intervals, or denominator uncertainty. NVML device energy is neither whole-system nor idle-subtracted energy; different caps need not optimize throughput and energy simultaneously.

This changes verification work but does not recover already executed draft work. Cost cells therefore distinguish (B, γ, G): a native static cell with $G = \gamma$ cannot substitute for a post-proposal cell with $G > \gamma$. The latter requires complete measured steps at that budget, or explicitly isolated, non-overlapping draft timings for a labelled replacement estimate; otherwise that action is uncalibrated. A zero speculative cap may still pay the full proposal cost. Pre-drafting selection is a separate implementation, and neural draft state must remain consistent across disabled and re-enabled steps.

Decision cost includes lookup, EWMA updates, IPC, and any synchronization introduced by telemetry. A constant-time lookup is not zero runtime overhead. Detailed route capture is a calibration and characterization facility; whether it must remain enabled online is an explicit ablation.

The native r02 CPU timestamps partially populate F6: across runs, the timed schedule-recording segment occupies 0.1285–0.3628% of host decode-step time, and acceptance updates occupy 0.0356–0.0707%. The control wrapper is nested within the schedule segment, so their durations cannot be added. These measured segments establish neither adaptive-selection cost nor total observer overhead, and are not subtracted from wall-clock results; both broader costs remain unmeasured.

Correctness boundary. Prefix truncation retains the original target verifier and sampler. This supports the intended sampling semantics only when bookkeeping, draft probabilities, KV state, and stopping conditions remain consistent. Unit checks and native greedy/sample tests validate the integration; a task or PPL spot check is a regression check, not a proof of distributional equality. We do not prune target experts in the main method.

6 Evaluation Structure

6.1 Comparisons and objectives

The primary objective is effective decode time per emitted token at the declared batch/load, with GPU joules per emitted token as a co-reported resource metric. For variable concurrency, request latency distributions and complete token throughput accompany that aggregate. GPU energy is device energy, not whole-system energy.

The comparisons use the same draft source and target kernels:

1. Ordinary decoding and native static $\gamma \in \{1, \ldots, 7\}$.
2. A tuned native threshold: an active cap below the selected batch cutoff and zero above it, including always-disabled and always-enabled cases.
3. A tuned native batch-to-γ table with no monotonicity restriction.
4. A Cascade-style utility controller, identifying which original mechanisms are reproduced or adapted [7].
5. The proposed batch-and-EWMA controller.

The static length, threshold and its active cap, and unrestricted table are selected on calibration data for the

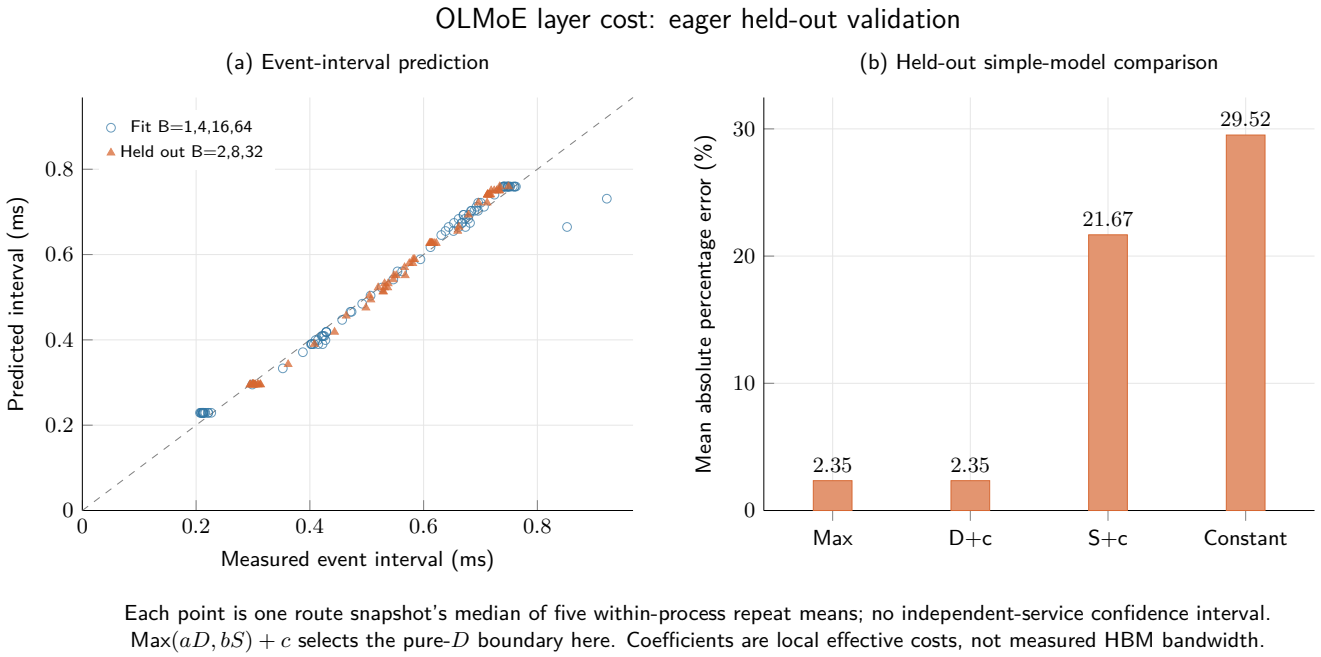


Figure 5: Single-layer eager cost validation, using measured expert counts. The 88 calibration snapshots and 66 held-out snapshots use disjoint nominal batch sizes. Each point is a median of five within-process repeat means, not an independent service run. The max-form and $aD + c$ give the same fit here; the inactive token-count branch is not evidence of zero compute cost.

same declared workload mixture and then frozen. Both threshold and table use native budget control; a full-draft-then-truncate threshold is only an execution-cost ablation and cannot replace the stronger native threshold. Per-cell best-static results on evaluation data are separately labelled hindsight references. At fixed B and stationary availability/acceptance, a tuned static length may already cover the useful action. Dynamic-load tests evaluate where online information can add value.

An offline best-configuration reference can show remaining headroom, but it is not automatically a causal per-step oracle. A policy changes subsequent requests and states. We do not splice favorable steps from different executions into a claimed achievable latency. Any model-based oracle is labelled with its counterfactual assumptions.

6.2 Measurement plan

The total campaign targets about 20 GPU hours. The measured component microbenchmark uses five warmed timing repeats per route snapshot with median reporting; these shared-process repeats are not independent service experiments. End-to-end configurations begin with two run repetitions, retaining raw values. Each report specifies warmup coverage, graph/eager mode, instrumentation, population and timing boundaries, failures, compilation, and run order. Two runs cannot establish a universal noise floor; claims are limited to the resolved effect and enlarged only where remaining uncertainty changes interpretation.

Availability, scheduled-prefix survival, unconditional accepted-prefix survival, and actual verification-shape distributions accompany throughput. Fixed-output experiments control output length; separate natural-stop checks align requests by source index and retain EOS, length-limit, failed, and unobserved outcomes. Fixed first-source samples support readable inspection. Token equality and prefix agreement are descriptive and do not by themselves establish task quality, a decoding bug, or distributional equivalence.

Energy uses the device's cumulative NVML energy counter when supported, otherwise approximately 50 ms power sampling and numerical integration over the exact measurement window. Sampling intervals, energy-counter units, idle treatment, and successful emitted-token denominators are recorded. Compile/startup energy is separate from steady-state generation. Current baseline and neural-drafter plans share the same physical device capacity and a declared 6 GiB KV allocation. Combined weight, metadata, and achieved KV memory must be reported; actual total resident allocation is not equalized by resizing KV.

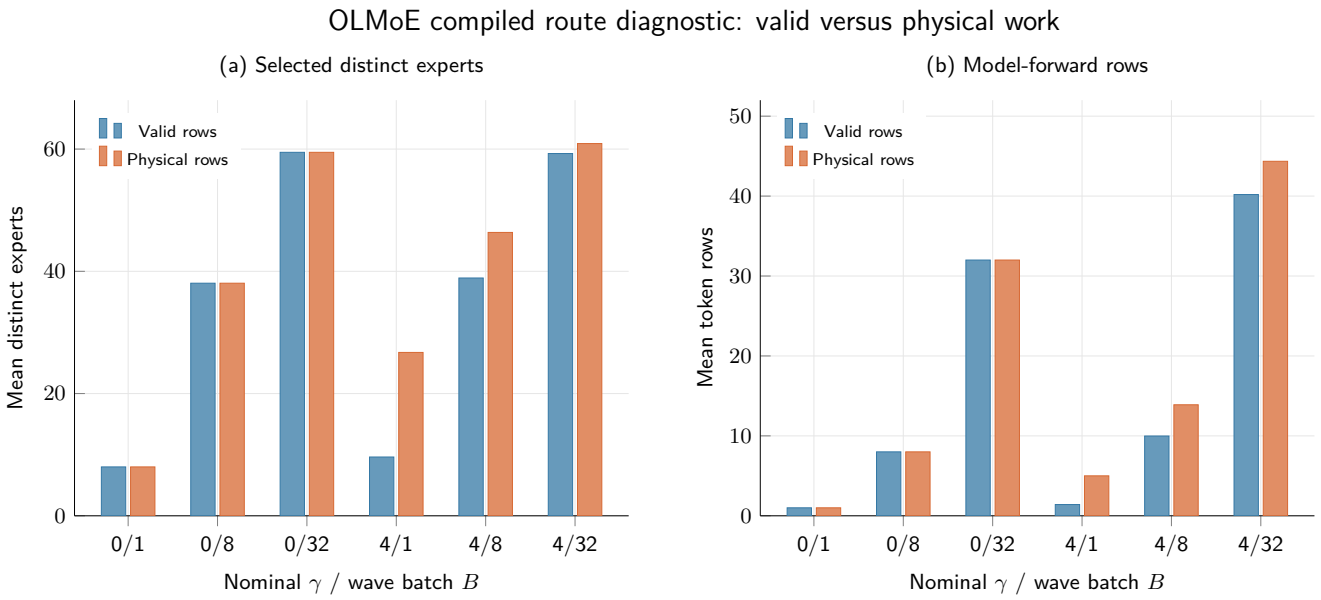


Figure 6: Instrumented compiled route diagnostic. Physical model-forward rows include graph padding and can activate experts beyond the valid-row union; expert matrix-multiplication tile padding is excluded. Means weight every decode layer-step equally, including shrinking wave tails. The six cases are correlated route observations, without independent confidence intervals or service-time inference.

Table 2: Evidence status after eager r02, component r03, and compiled route diagnostics. Component timing and instrumented physical participation do not establish compiled service or controller benefits.

Output	Evidence and remaining question
F1	Actual-shape occupancy and layer cost measured on respective holdouts; compiled valid/physical unions diagnosed.
F2	Full 7×8 eager finite-wave surface measured; compiled service and continuous-load comparisons pending.
F3	Increment over tuned static and batch schedules; latency tradeoffs.
F4	Device energy per token measured for native static caps; adaptive-policy energy pending.
F5	Availability/emission measured; offline tracking diagnosed; online controller validation pending.
F6	Native CPU segments timed; adaptive selection, total observer cost, and discarded draft work pending.
T1	Predicted versus observed transition intervals, with uncertainty.
T2	Action and information comparison with nearest methods.

6.3 Evidence map and remaining comparisons

Measurement pending: Complete native controller comparisons, compiled service performance, neural-drafter experiments, and natural-generation inspection. Current measurements cover one exploratory target/drafter/workload setting, not a completed evaluation.

6.4 Limits of transfer

Matching GPU instruction-set support does not establish matching performance. Bandwidth, cache, clock/power limits, available memory, kernel selection, and quantization can shift the transition. Each GPU/precision pair needs its own calibration. Additional models test a specific prediction about routing, E/k, or fixed cost; their count alone does not establish generality.

7 Related Work

MoESD is the nearest explanation and model baseline [2]. We build on its batch-dependent expert activation account. Its published modeling algorithm fits performance parameters; our inspection did not locate a MoESD online optimal-length controller. That limited observation does not establish absence of any unpublished implementation.

Cascade already implements adaptive MoE speculation in vLLM using utility measured across test-and-set phases, including disabling, back-off, and hill-climbing [7]. Our intended difference is explicit batch state and route-calibrated cost, evaluated against its feedback-driven approach. This is an empirical question rather than an assumed advantage.

AdaEAGLE predicts draft length with a learned lightweight predictor [10]. SpecDec++ predicts conditional acceptance and stops according to accumulated rejection risk [1]; DISCO uses a shallow classifier for draft continuation [4]. They motivate adaptive length but differ in information and training cost. EAGLE-3 is a draft-generation mechanism, not evidence for the value of our controller [3].

Native vLLM dynamic schedules make batch-only adaptation an existing engine capability [8, 9]. Finally, MoE-Spec changes verification expert budgets and permits a quality tradeoff [5]; our main method keeps target routing intact. The optional expert-dropping extension is not part of the present claims.

8 Conclusion

The OLMoE/n-gram experiments show why draft cap, useful progress, expert occupancy, and service cost must be measured separately. Actual verification shape improves the occupancy approximation in held-out cap executions; compiled padding can further enlarge the physical expert union. A distinct-expert cost fit predicts the sampled single-layer eager interval, without identifying a compute transition or full-model fixed-cost dominance. Finite-wave throughput, request-level effective token latency, and energy give different views of the same executions. These findings guide uninstrumented compiled measurements and broader draft coverage before attributing the performance surface or claiming an adaptive-policy benefit. The manuscript remains an exploratory determinants characterization; pending controller and additional-target experiments must establish broader scope and decision value.

References

[1] Kaixuan Huang, Xudong Guo, and Mengdi Wang. SpecDec++: Boosting speculative decoding via adaptive candidate lengths. In *Conference on Language Modeling*, 2025. arXiv:2405.19715v3.

[2] Zongle Huang, Lei Zhu, Zongyuan Zhan, Ting Hu, Weikai Mao, Xianzhi Yu, Yongpan Liu, and Tianyu Zhang. MoESD: Unveil speculative decoding's potential for accelerating sparse MoE. In *Advances in Neural Information Processing Systems*, 2025. arXiv:2505.19645, version 4, revised February 16, 2026.

[3] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. EAGLE-3: Scaling up inference acceleration of large language models via training-time test. In *Advances in Neural Information Processing Systems*, 2025.

[4] Jonathan Mamou, Oren Pereg, Daniel Korat, Moshe Berchansky, Nadav Timor, Moshe Wasserblat, and Roy Schwartz. Dynamic speculation lookahead accelerates speculative decoding of large language models. In *Proceedings of the 4th NeurIPS Efficient Natural Language and Speech Processing Workshop*, volume 262 of *Proceedings of Machine Learning Research*, pages 456–467, 2024.

[5] Bradley McDanel, Steven Li, Sruthikesh Surineni, and Harshit Khaitan. MoE-Spec: Expert budgeting for efficient speculative decoding. arXiv:2602.16052v1, 2026.

[6] Safe AI Lab. EAGLE: Official implementation and checkpoint catalog. https://github.com/SafeAILab/EAGLE, 2026. Accessed October 16, 2026; model compatibility must be verified per checkpoint.

[7] Anish Saxena, Po-An Tsai, Hritvik Taneja, Aamer Jaleel, and Moinuddin Qureshi. Utility-driven speculative decoding for mixture-of-experts. arXiv:2506.20675v1, 2025.

[8] vLLM contributors. Dynamic speculative decoding: Documentation at v0.26.0. https://github.com/vllm-project/vllm/blob/v0.26.0/docs/features/speculative_decoding/dynamic_speculative_decoding.md, 2026. Accessed October 16, 2026.

[9] vLLM contributors. Speculative configuration and dynamic schedule implementation at v0.26.0. https://github.com/vllm-project/vllm/blob/v0.26.0/vllm/config/speculative.py, 2026. Configuration and v1/spec_decode/dynamic/utils.py; accessed October 16, 2026.

[10] Situo Zhang, Hankun Wang, Da Ma, Zichen Zhu, Lu Chen, Kunyao Lan, and Kai Yu. AdaEAGLE: Optimizing speculative decoding via explicit modeling of adaptive draft structures. arXiv:2412.18910v1, 2024.